

See discussions, stats, and author profiles for this publication at: <https://www.researchgate.net/publication/373252433>

An Approach for Visualizing SVG Sprites Systems

Chapter · August 2023

DOI: 10.1007/978-3-031-33261-6_34

CITATIONS

2

READS

65

2 authors:



Nikolay Buhtiyarov

University of Library Studies and Information Technologies

6 PUBLICATIONS 15 CITATIONS

SEE PROFILE



Daniela Borissova

Institute of Information and Communication Technologies, Bulgarian Academy of...

148 PUBLICATIONS 946 CITATIONS

SEE PROFILE

An Approach for Visualizing SVG Sprites Systems



Nikolay Buhtiyarov  and Daniela Borissova 

Abstract The contemporary design of Internet applications relies on a variety of icons. This requires special attention to the format of the images used as icons. It is important that these images are not only of good resolution but also of sufficiently high quality. These basic requirements are met by SVG format and images could be organized in sprites with different numbers of icons. Due to the huge amount of used icons, it is of high importance not only to organize the icons into SVG sprites but also to be properly visualized. In this regard, the current article deals with the problem of automation of the process of visualizing the SVG sprites. A three-stage approach based on JavaScript algorithms able to organize and visualize multiple SVG sprites with different numbers of icons is proposed. The conducted tests approve the applicability of the proposed approach for visualizing SVG sprites with different numbers of items.

Keywords SVG · Sprite system · Algorithms · JavaScript

1 Introduction

The modern design of various web applications requires collaboration between all teams involved in the development of the particular product. The concept of modular programming could be used to provide the needed foundation to design an application rapidly thus enables teams to not only work concurrently, but also makes testing, deployment, and maintenance easier for both development companies and the business [1, 2]. On the other hand, modular development allows individual modules to be reused for other applications, saving valuable development time. Time to market

N. Buhtiyarov · D. Borissova (✉)

Institute of Information and Communication Technologies at the Bulgarian Academy of Sciences,
1113 Sofia, Bulgaria

e-mail: daniela.borissova@iict.bas.bg

N. Buhtiyarov

e-mail: nikolay.buhtiyarov@iict.bas.bg

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2023

397

Á. Rocha et al. (eds.), *Information Technology and Systems*,

Lecture Notes in Networks and Systems 692,

https://doi.org/10.1007/978-3-031-33261-6_34

plays an important role in product development, and design teams need to streamline the process without compromising the user experience. The user interface (UI) is a key component of any interactive software application and it is important for software developers to be focused on UI design using the proper tools [3]. Many model-based UI development environments have been proposed that allow semi-automated UI source code generation by using models with different levels of abstraction [4–6]. In some cases, web applications need to contain different information according to the particular problem, and UI is generated on the fly [7].

While designing the UI, it is good to follow a consistent design that is also intuitive design. Visual consistency relies on similar elements such as fonts, sizes, icons, etc. being perceived in the same way. Therefore, typography, colors, spacing, grid, size, and positions must be defined uniformly. The availability of consistent visual elements will contribute to a better user experience [8]. The main aspect of UI design in the modern internet space are icons. They appear on every website and application on the Internet, from navigation menus to social icons and indicators. Icons as carriers of information are an important feature of UI [9]. That is why some authors try to automate process for icon designs assisting in UI development by developing a new tool [10]. Their popularity shows a constant interest from the developers of various applications. As a result, every developer is faced with the dilemma of choosing which icon set to use. On the other hand, the massive use of mobile devices and the advent of high pixel density displays require sophisticated visual assets to render images in the appropriate format [11]. It is worth to mention also the flexibility of JavaScript [12] and its advantage to create variety of web-based modular applications due to the compatibility with variety of browsers [13].

Until recently, developers had to rely only on image formats to meet the requirements of the developing applications. The used images as icons result in low quality and resolution. This problem is solved today by the two solutions available: icon fonts or SVG icons (scalable vector graphics). Both icon formats have proponents who argue the merits of one over the other [14]. Icon fonts that contain symbols and glyphs instead of letters or numbers were introduced in 2012. A comparison between icon fonts and SVG with respect to size, accessibility, performance, scalability, deployment, and cross-browser compatibility can be found in [15]. Recently, there is widespread acceptance of SVG icon format due to its better performance, higher standards of accessibility, high rendering quality, unparalleled flexibility, and complete customization. The SVG format also supports dynamic changes, can be used to create interactive documents, and animations can be performed using declarative animation functions or by using a script [16]. It is worth to be mention also the fact, that De-coupling the SVG visualization specification from SVG rendering, and obtaining a dynamic VDOM, adds flexibility and opportunity for visualization system research [17]. Last but not least, it should be mentioned the fact that using SVG leads to a reduction in the number of HTTP requests to replace images [18].

User interface design work falls to developers, visually oriented interface designers, or subject matter experts. Therefore, each tool will be of great importance in speeding up the application development process. Such a tool that contributes for speed up website loading is the sprite. Image sprite represents a collection of

individual images all participating in the formation of a single image. Picture Sprite compiles multiple icons into a single file that is used by designers to improve website performance. In this regard, the current article deals with the problem of visualizing sets of sprites composed of multiple icons. The essence of the proposed approach is the possibility developer to be informed about the overall number of icons, the number of icons that compose each sprite, and the ID of each icon.

The rest of the article is organized as follows: Sect. 2 provides a description of the problem, Sect. 3 describes the essence of the proposed approach for previewing all icons from SVG sprites, Sect. 4 contains some results and discussion and the conclusions are drawn in Sect. 5.

2 Problem Description

SVG images can be previewed with most graphic software, even with web browsers. When many SVG files are combined into SVG sprite, their initial state is to be invisible, and only a single icon can be displayed, by referring to it by ID. This great advantage is also SVG sprites' biggest disadvantage because it leads to an inability to preview the collection of icons. From the development point of view, this is a huge drawback. The inability to preview the icons from the sprite in most cases leads to icon repetition. In reality, the sprites we are working with contain about 30% repeated icons. One way to preview the icons is to create an HTML file where to hardcode every icon from the sprite. But this solution will be obsolete soon after its implementation because requires additional work every time new icons are added or removed.

The only other solution by the time available on the internet was provided by a highly reputable website for front-end web development "CSS Tricks" [19]. JavaScript code that selects all "symbol" tags from the SVG sprite, and then will add each icon to separate SVG tag to the DOM. The same as the HTML example but automated with JavaScript. At first glance much better solution, but it is the same effort because, in order to work, the developer needs every time to copy the content of the sprite, paste it into the HTML file, and then run the script. In our case study, there are 12 sprites with 600 + icons combined. Although this was not the solution we were looking for, it was helpful and gave a starting point for the concept of what needs to be done, to create easy to use dynamic preview system.

3 Description of the Proposed Approach for Previewing all Icons from SVG Sprites

The proposed approach for previewing all icons from SVG sprites can be realized in three main stages as shown in Fig. 1:

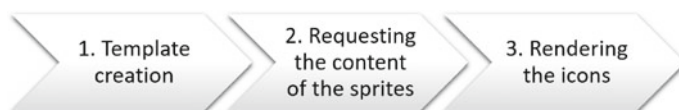


Fig. 1 Main stages of the proposed approach

The steps could be realized by adequately coding a single HTML file containing the HTML, CSS, and JS codes. This file needs to be located in the directory that contains the SVG sprites. CSS code will not be discussed because it only affects the design.

3.1 *Template Creation*

Following the scheme in Fig. 1, in the first stage, it is needed to create an empty container to store the content of the SVG sprites. For the goal, it is needed to create 2 templates: one of them represents the section that will contain the icons of each sprite, while the second one will be used to represent the container of a single icon. The corresponding codes for these templates are as follows:

```

<div class="sprites-container"></div>
(Template "Section")
<template id="section">
  <section>
    <h2></h2>
    <div class="icons-container"></div>
  </section>
</template>

(Template "Figure")
<template id="figure">
  <figure>
    <div class="icon-container">
      <svg width="" height="">
        <use xlink:href=""></use>
      </svg>
    </div>
    <figcaption></figcaption>
  </figure>
</template>

```

A wrapper container that contains the general information such as title, informative texts and input field that is used for dynamic resizing of the icons can be added

for additional functionality and information to the user. The HTML templates are mandatory to be able to preview the sprites. They will be populated dynamically with data provided by JavaScript in order for the icons to be visualized.

3.2 *Requesting the Content of SVG Sprites*

The second stage requires to be performed a requesting the content of SVG sprites. This activity could be realized via two JavaScript functions. The aim of the first function is to send requests to the SVG sprites files and adds their content to the DOM tree. The second JavaScript function is needed to render the icons. The code of these functions is listed below:

```
// Sends requests to the SVG files and adds them to the
DOM
function addSvgSpritesToDOM() {
  sprites.forEach(sprite => {
    const req = new XMLHttpRequest();
    req.open("GET", `./${sprite}.svg`, true);
    req.send();
    req.onload = function(e) {
      const div = document.createElement("div");
      div.classList.add(sprite);
      div.innerHTML = req.responseText;
      spritesContainer.appendChild(div);
    }
  })
}

// Observes when all SVG files are added to the DOM, then
initiates the rendering of the icons
const observer = new MutationObserver(mutations => {
  mutations.forEach(record => {
    if (spritesContainer.childElementCount ===
sprites.length) {
      renderIcons();
    }
  })
})

observer.observe(spritesContainer, {
  childList: true
})
```

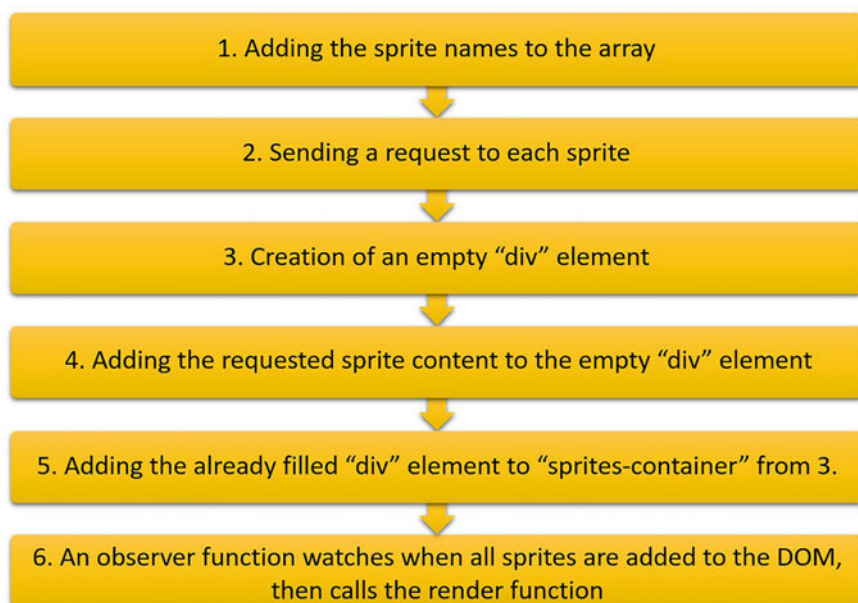


Fig. 2 Algorithm for requesting the content of SVG sprites

The used algorithm behind these functions is realized by the following 6 steps as shown in Fig. 2:

Knowing the names of the sprite files provided from the array, the "addSvgSprintedToDOM" function will send a request to each file, extracting its text content and adding it to the DOM. The same as manually copy/paste the content, but automated! At this stage, the icons are not visible. The other code observes when all the sprites are finally added to the DOM and then calls the "renderIcons" function. This approach guarantees the reliability of the application, no matter the number of sprites and icons are provided, it will execute properly every time.

3.3 Rendering the Icons

The third stage is intended to render the icons. This activity can be implemented by another JavaScript function:

```

function renderIcons() {
    sprites.forEach(sprite => {
        const svgSprite = document.querySelec-
tor(`.${sprite}`);
        const symbols = svgSprite.querySelectorAll("sym-
bol");
        const section = document.querySelector("#sec-
tion").content.cloneNode(true);
        const heading = section.querySelector("h2");
        const iconsContainer = section.querySelec-
tor(".icons-container");
        const spriteText = sprite.charAt(0).toUpperCase()
+ sprite.slice(1);
        heading.innerHTML = `${spriteText}.svg sprite
(${symbols.length} icons)`;

        symbols.forEach(symbol => {
            const figure = document.getElementById("fig-
ure").content.cloneNode(true);
            const iconContainer = figure.querySelec-
tor(".icon-container");
            const svg = figure.querySelector("svg");
            const use = svg.querySelector("use");
            const figCaption = figure.querySelector("fig-
caption");
            figCaption.innerHTML= symbol.id;
            svg.setAttribute("width", iconSize);
            svg.setAttribute("height", iconSize);
            use.setAttribute("xlink:href",
`/${sprite}.svg#${symbol.id}`);
            iconsContainer.appendChild(figure);
        });
        iconsCount += symbols.length;
        wrapper.appendChild(section);
    })
    iconsCountEl.innerHTML = iconsCount;
    changeIconsSize();
}

```

The used algorithm behind the function “renderIcons” is realized by the following 8 steps as shown in Fig. 3:

At this stage, the icons can be visualized within the web browser environment. The results obtained using the described above algorithms that are appropriately implemented as JavaScript functions are presented in the next section.

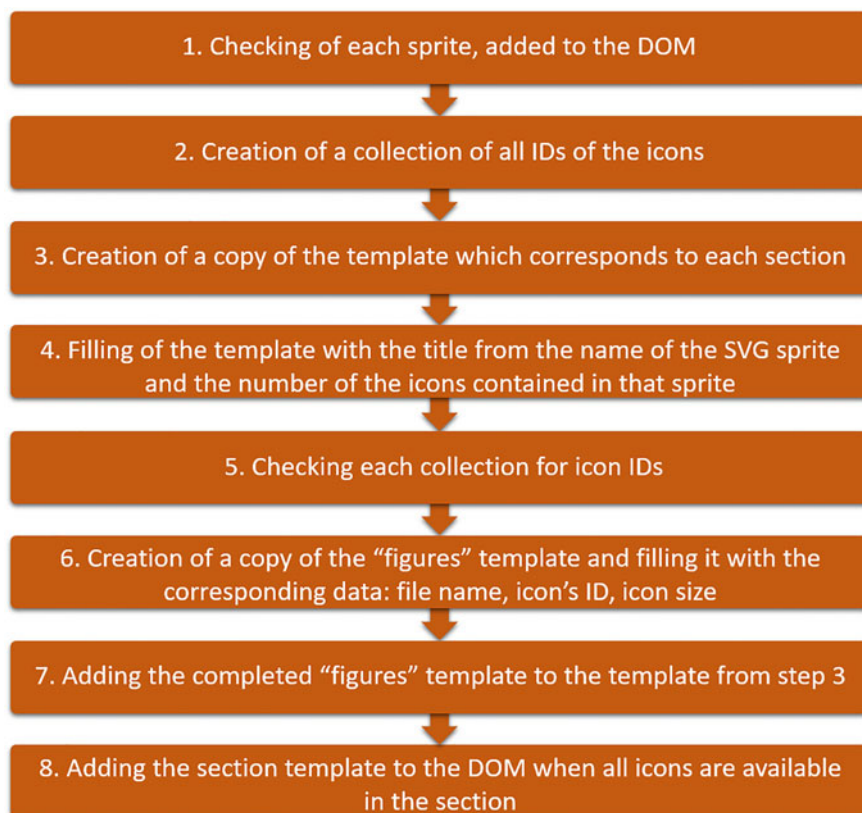


Fig. 3 Algorithm for rendering the icons

4 Results and Discussion

To send the HTTP requests for SVG sprite icons preview, a simple web server is needed to serve the HTML file. The server used for this research is "Live Server", which can be installed as NPM package [20]. Considering the fact developers already work on a WEB app, the chances to have installed Node.js and NPM already are almost 100%, the only thing needs to be done is to execute the command "npx" live-server". To simplify this process even further, a batch script file with this starting command is added.

The described approach has been tested in practice for visualizing a set of 12 sprites with more than 250 icons. The final result we are looking for about the described problem is presented in detail in the repository GitHub [21]. Let's see what is going on, why this is the only working solution, you can find on the internet, and how with a single click of the mouse you can get such beautiful collection of icons. The corresponding result is shown in Fig. 4.

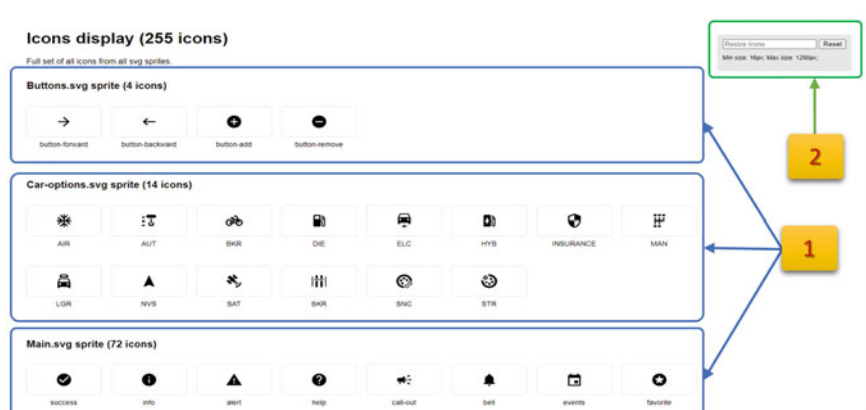


Fig. 4 SVG sprites visualizing: (1) block of single sprite; (2) menu for changing the size of icons within sprites

It could be seen that each SVG sprite is visualized with following attributes: name of the sprite and number of the icons within each sprite. For example, the sprite “*buttons.svg*” contains only four icons unlike to “*main.svg*” sprite that contains 72 icons. The default size of all icons is $32px$. Some icons might have different viewport, or this size is too small to see all needed details. For this reason, an additional menu (see position #2 in Fig. 4) is included. This menu allows to change the icons dimensions within interval from $16px$ to $1260px$. Thanks to this functionality, it is possible to see in detail each icon included within a given sprite.

In the end, to summarize all the effort, only one developer needs to do the initial set-up process by adding the names of all sprites to the array in the HTML file. A double click on the “*Show icons.cmd*” file will preview all icons. This approach is 100% dynamic and when we add new icons to the sprites, they will be automatically displayed, the same as when icons are removed.

5 Conclusions

In the modern design of Internet applications, there is an increasing tendency to use a large variety of icons. This requires paying particular attention to the format of the images used as icons. Along with this, it is essential these images are not only of good resolution but also of sufficiently high quality and be able to load quickly. These basic requirements are completely satisfied by the SVG format which is the preferred choice for most UI developers.

Of particular importance are SVG sprites composed of many icons. In the current article an approach to visualize such SVG sprites is proposed. This approach is realized following three main stages concerning: 1) template creation; 2) requesting

the content of the sprites and 3) rendering the icons. The first stage is realized by using HTML, while the second and third rely on JavaScript. The described approach contributes to the automation of the process of icon previewing. The proposed solution can easily be updated by adding/removing icons or new sprites. The conducted tests show the applicability of the proposed approach to visualizing given sets of sprite systems. Thanks to well-designed software modules for SVG sprites visualization, it is possible to easily extend them as well as reuse them. These circumstances are a prerequisite for achieving good resilience, maintenance, and adaptation.

Acknowledgements This work is supported by the Bulgarian National Science Fund by the project “*Mathematical models, methods and algorithms for solving hard optimization problems to achieve high security in communications and better economic sustainability*”, KP-06-N52/7/19-11-2021.

References

1. Seydnejad S (2016) Modular Programming with JavaScript. Packt Publishing, Birmingham
2. Mustakerov IC, Borissova DI (2013) Data structures and algorithms of intelligent web-based system for modular design. <https://doi.org/10.5281/zenodo.1086885>
3. Ruiz J, Asensio ES, Snoeck M (2020) Learning UI functional design principles through simulation with feedback. *IEEE Trans Learn Technol* 13:833–846. <https://doi.org/10.1109/TLT.2020.3028596>
4. Tran TAV, Vanderdonckt J, Kolp M, Wautelet Y (2010) Generating user interface for information applications from task, domain and user models with DB-USE. In: 1st International workshop on user interface extensible markup language UsiXML’2010, pp 183–194
5. Ruiz J, Serral E, Snoeck M (2019) Evaluating user interface generation approaches: model-based versus model-driven development. *Softw Syst Model* 18:2753–2776. <https://doi.org/10.1007/s10270-018-0698-x>
6. Kennard R, Leaney J (2010) Towards a general purpose architecture for UI generation. *J Syst Softw* 83(10):1896–1906
7. Dimitrova Z, Borissova D, Dimitrov V (2021) Design of web application with dynamic generation of forms for group decision-making. In: Saeed K, Dvorsky J (eds) *Computer information systems and industrial management. CISIM 2021. Lecture notes in computer science*, vol 12883, pp 112–123. https://doi.org/10.1007/978-3-030-84340-3_9
8. Hinderks A, Mayo FJD, Thomaschewski J, Escalona MJ (2022) Approaches to manage the user experience process in agile software development: a systematic literature review. *Inf Softw Technol* 150:106957. <https://doi.org/10.1016/j.infsof.2022.106957>
9. Black A (2017) Icons as Carriers of Information. *Information Design: Research and Practice*, pp 315–329
10. Feng S, Ma S, Yu J, Chen C, Zhou T, Zhen Y (2021) Auto-Icon: an automated code generation tool for icon designs assisting in UI development. In: 26th International Conference on Intelligent User Interfaces (IUI’21), April 2021, pp 14–17. <https://doi.org/10.1145/3397481.3450671>
11. Cryer J (2015) Using grunt with Images, SVG, and Icons. In: *Pro Grunt.js*. Apress, Berkeley, CA, pp 85–100. https://doi.org/10.1007/978-1-4842-0013-1_5
12. Kantor I (2022) The Modern JavaScript Tutorial. <https://javascript.info/>
13. Mustakerov I, Borissova D (2013) Data structures and algorithms of intelligent Web-based system for modular design. *Int Schol Sci Res Innov* 7(7):876–881. <https://doi.org/10.5281/zenodo.1086885>

14. Colton: An Optimized Icon System – SVG Sprites, (2021). <https://medium.com/the-crazy-coder/an-optimized-icon-system-svg-sprites-5067e006a770>. Accessed 11 July 2022
15. Nikhil: Icon Fonts vs SVG – Clash of the Icons, (2019). <https://www.lambdatest.com/blog/its-2019-lets-end-the-debate-on-icon-fonts-vs-svg-icons/>. Accessed 07 July 2022
16. Scalable Vector Graphics (SVG) 2. <https://www.w3.org/TR/2016/CR-SVG2-20160915/single-page.html>. Accessed 01 July 2022
17. Schwab M, et al (2022) Scalable scalable vector graphics: automatic translation of interactive SVGs to a multithread VDOM for fast rendering. *IEEE Trans Visual Comput Graph* 28(9):3219–3234. <https://doi.org/10.1109/TVCG.2021.3059294>.
18. Shivakumar SK (2020) *Modern Web Performance Optimization: Methods, Tools, and Patterns to Speed Up Digital Platforms*. Apress, New York
19. Coyier CH (2019) A snippet to see all SVGs in a sprite. <https://css-tricks.com/a-snippet-to-see-all-svgs-in-a-sprite/>. Accessed 16 June 2022
20. An introduction to the Npm package manager. <https://nodejs.dev/en/learn/an-introduction-to-the-npm-package-manager/>. Accessed 12 July 2022
21. Buhtiyarov N Icons display. <https://github.com/nikolaybuhtiyarov/icons-display>. Accessed 02 Sep 2022